THE LOGIC MACHINES

Classical, many-valued and modal logic, with the wiring showing

A logic is not a discovery about how reasoning must go.
It is a machine, with settings, and you can see all of them.

WHO THIS IS FOR	Anyone who can read a table and is willing to hold a pencil. No mathematics beyond counting is needed. No prior logic is assumed.
THE PROMISE	Every rule in this book is computed in front of you. Nothing is asserted. If a table says something, you can rebuild that table yourself from the definition above it.
THE THREE DIALS	Every logic in this book is the same machine with three settings: **V**, the states it may be in; **G**, how those states combine or move; and **theta**, the constraint saying which configurations are admissible. Modal logic is not a fourth dial. It is theta pointed at a network of situations instead of at a set of truth values.
THE WARNING	Part Five is about what these machines cannot tell you, including the most common mistake made with the strongest of them. A logic being coherent inside itself says nothing about whether the world is shaped like it.

PART ZERO

THE INSTRUMENT

Three dials, and one habit that makes the rest of the book easy.

What a logic actually is

Strip away the symbols and a logic is three decisions somebody made.

V	the value space	What states the machine is allowed to be in. Two of them, usually. It does not have to be two, and in modal logic they are not truth values at all.
G	the gradient	How those states combine or move. What AND does, what OR does, what NOT does. In modal logic, what it means to step from one situation to another.
theta	the threshold	**The constraint.** Which configurations count as admissible. This one is almost never mentioned and it decides more than the other two.

Change any one of those three and you have a different machine, with different rules coming out of it. Nobody discovered those rules. **They fall out of the settings.**

Theta is the one to watch

It is worth saying now what theta does, because it wears two different costumes in this book and people who do not notice think they are two different subjects.

in a many-valued logic	theta is a **cut on the values**: which of them count as good enough to assert. The standard books call this the **designated set**, written D, and it is the same dial.
in a modal logic	theta is a **cut on the arrows**: which networks count as admissible. Demand that every dot sees itself and you have named a system.

Both are the same move. Theta says which configurations are in and which are out. Turn it on the values and you get K3 or LP. Turn it on the arrows and you get T or S4 or S5. There is no fourth dial in this book, and modal logic is not a different kind of machine.

The habit

Whenever a rule of logic feels like it must be true, ask what would have to be the case for it to fail. That question is doing the same job the pencil does in arithmetic: it takes a feeling and turns it into something you can check.

On load

Nothing in this book is hard. Some of it asks you to hold four or five things at once, which is a different problem with a different fix.

A truth table is a **load-relief device**. It exists so that the cases sit on the paper instead of in your head. When somebody says logic is difficult, what is usually happening is that they are trying to keep four rows in memory and one keeps falling out.

THE ONE INSTRUCTION FOR THE WHOLE BOOK

Write the table out. Every time.

It costs about twenty seconds. It removes almost every mistake anybody makes in this subject.

The people who are good at logic are not holding more in their heads. They are writing more down, faster, and with a tidier layout.

PART ONE

THE TWO-VALUE MACHINE

Classical logic. Two settings, four connectives, and everything else computed from them.

The claim

A **claim** is a sentence that can be true or false. That is the only thing this machine handles.

The door is open.	a claim. It is one or the other.
Open the door.	not a claim. An order is not true or false.
Is the door open?	not a claim. A question is not true or false.
This sentence is false.	trouble. Part Five.

The first setting: V has two values

$$V = \{ T , F \}$$

Two values, and every claim gets exactly one. **This is a decision, not a fact.** Part Two turns this dial and the machine still works.

NOT: the flip

NOT takes a value and gives the other one.

a	NOT a
T	F
F	T

Two rows because there are two things a can be, and a table with every row is a **complete check**. Nothing is left to imagine.

Do it twice and you are back where you started, which is the same turning-round-twice you already know.

AND: strictest wins

a AND b is true only when both are.

a	b	a AND b
T	T	T
T	F	F
F	T	F
F	F	F

Four rows, because two values with two slots gives two times two. Count the rows before you write them and you will never lose one.

OR: easiest wins

a OR b is true when at least one is. This is **inclusive** or: both counts as yes.

a	b	a OR b
T	T	T
T	F	T
F	T	T
F	F	F

Ordinary speech often means the other one. 'Soup or salad' usually means not both. That is **exclusive** or, it is a different machine, and logic picked inclusive because it composes better. The choice is written down rather than assumed, which is the whole point of doing this on paper.

The same machine, written as arithmetic

Everything above can be written without the words T and F, and doing so removes most of the confusion people have with this subject.

Write **false as 0** and **true as 1**. Then every connective is an ordinary arithmetic operation you already know.

in words	in arithmetic	check it
NOT a	**1 - a**	1-0 = 1, and 1-1 = 0
a AND b	**the smaller one**, min(a, b)	and on 0 and 1 that is exactly **a times b**
a OR b	**the bigger one**, max(a, b)	0 or 1 gives 1, because 1 is bigger

AND is your zero and one times table. Anything times zero is zero, which is why one false input kills the whole conjunction. You have known that since Part One of the arithmetic book.

Check AND both ways and they agree in all four rows:

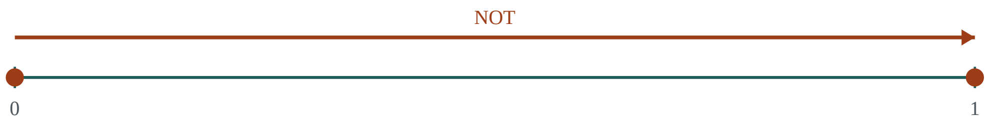
a	b	min(a,b)	a x b
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

NOT is the flip you already know

1 - a is the same move as minus on a number line: it turns you round. Do it twice and you are back where you started.

$$1 - (1 - a) = a$$

That is 'two negatives make a positive', arriving in logic. Not a new rule. The same one.



The point of writing it this way is not elegance. It is that 0 and 1 carry no meaning, so nothing can be smuggled in. The machine only ever saw two numbers, and it never saw the moon.

De Morgan: the two rules that are one rule

What is NOT (a AND b)? Build the table and read it off.

a	b	a AND b	NOT(a AND b)	NOT a	NOT b	NOTa OR NOTb
T	T	T	F	F	F	F
T	F	F	T	F	T	T
F	T	F	T	T	F	T
F	F	F	T	T	T	T

The last two columns match in every row. That is not a rule somebody imposed; it is something you just measured.

`NOT (a AND b) is the same as NOT a OR NOT b`

`NOT (a OR b) is the same as NOT a AND NOT b`

Said in English: to break 'both', you only need one to fail. To break 'either', you need both to fail. Once you hear it that way you will never need to look it up.

The conditional: a promise with one breaking case

If a then b is the one people find strange, and it is strange for a reason worth naming rather than glossing.

Treat it as a **promise**. I say: if it rains, I will bring an umbrella. When have I lied to you?

a	b	what happened	if a then b
it rains	I bring one	promise kept	T
it rains	I do not	**promise broken**	**F**
no rain	I bring one	I did not lie. I said nothing about dry days	T
no rain	I do not	I did not lie. Nothing was promised	T

Exactly one row is false. True, then false. That is the whole definition and everything odd about the conditional comes from those last two rows.

a	b	if a then b
T	T	T
T	F	F
F	T	T
F	F	T

Why the last two rows are like that

If it does not rain, the promise was never tested. An untested promise is not a broken one, so it counts as kept.

This is called **vacuous truth**, and here is the famous example that confuses everybody:

```
If the moon is cheese, then 2 + 2 = 5. TRUE
```

Almost everyone's first response is: **but the moon isn't cheese, and the two things have nothing to do with each other.**

That reaction is completely correct about the world and completely beside the point about the machine. Here is why, and the fastest way to see it is to throw the words away.

The same sentence, in numbers

'The moon is cheese' is false, so it is **0**. '2 + 2 = 5' is false, so it is **0**. That is everything the machine gets. It never sees a moon.

```
0 -> 0 = max(1 - 0, 0) = max(1, 0) = 1
```

One. True. And there is nothing to argue with, because it is arithmetic on two numbers.

There is a second way to read the conditional that makes it even plainer. Check the table: **a -> b is 1 exactly when a is less than or equal to b.**

a	b	a -> b	is a <= b ?
0	0	1	yes, 0 <= 0
0	1	1	yes, 0 <= 1
1	0	**0**	**no, 1 is not <= 0**
1	1	1	yes, 1 <= 1

So the conditional is a **comparison**. It asks one question: is the left number no bigger than the right one? Nothing else.

`moon is cheese = 0. 2+2=5 is 0. Is 0 <= 0? Yes.`

Nobody objects that 3 is less than 7 even though threes and sevens have nothing to do with each other. A comparison does not need its two sides to be related. The conditional is a comparison.

What you were presuming that the machine was not

The objection 'but they have nothing to do with each other' is assuming a **fourth** thing that classical logic does not have.

V	two values	the machine has this
G	how they combine: 1-a, min, max	the machine has this
theta	which value counts as assertible	the machine has this
relevance	the two sides must be about the same subject	**the machine does not have this, and never claimed to**

Relevance is not a rule classical logic broke. It is a setting that was never installed. The machine has no way to represent 'these two claims are about related things', because a claim is a single number and a number is not about anything.

THE MACHINE THAT DOES HAVE THAT DIAL

Relevance logic adds it. The setting is a tax on the conditional: **every premise must contribute a variable**. The two sides must share content.

Under that filter, 'moon is cheese' and '2 + 2 = 5' share nothing, so the conditional is refused before any arithmetic happens.

And 'if A then A' shares trivially, so it is kept.

Nothing about the tables changed. A filter was added in front of them, and it is a setting of theta: which derivations count as admissible.

So the honest description of the cheese sentence is: **true in a machine with no relevance filter, refused in a machine that has one**. Both machines are consistent. They are answering different questions.

Why classical logic left it out

Not by oversight. Leaving relevance out is what makes the conditional **a function of the two values alone**, and that is worth a great deal:

it is decidable by table	four rows and you are done. Relevance needs you to inspect what the claims are about
it composes	you can chain conditionals without checking subject matter at every link
it is what circuits do	a logic gate sees two voltages. It has no access to what they mean, and it works anyway

The price is exactly the cheese sentence. You get a machine that is fast, checkable and buildable in silicon, and it will cheerfully tell you true things that are completely useless. **That is the trade, and it is usually worth it.**

So the sentence is true and it tells you **absolutely nothing**, and those two facts are compatible. The promise was never tested, so it was never broken.

VACUOUS TRUTH IS A REAL ENGINEERING HAZARD

This is not a philosopher's curiosity. It causes actual failures.

A safety check reads: **every valve rated above 200 bar must be inspected.** The report comes back: all conditions met.

But there were **no valves above 200 bar** in the batch. The condition was never tested on anything. It passed by having nothing to fail on.

The same shape ruins software tests. 'For every user in the list, check the password is strong.' If the list is empty, every check passes and the report is green.

The fix is one number: how many cases actually got tested? If the answer is zero, the pass carries no information. Ask for the count, always.

A green tick over an empty box looks exactly like a green tick over a hard-won result. The only way to tell them apart is to ask how many things were checked.

The other three shapes

From 'if a then b' you can build three related sentences, and two of them are traps.

original	if a then b	if it rains, the ground is wet	given
converse	if b then a	if the ground is wet, it rained	**NOT the same.** A burst pipe wets the ground
inverse	if not a then not b	if it does not rain, the ground is not wet	**NOT the same.** Same pipe
contrapositive	if not b then not a	if the ground is not wet, it did not rain	**the same, always**

Only the contrapositive is guaranteed. Check it in the table: the original is false in exactly one row, T then F. The contrapositive is false in exactly one row, NOT b true and NOT a false, which is b false and a true. **Same row.** They cannot come apart.

Swapping a conditional round is the single most common mistake in public reasoning. Most tests are read this way: 'the test is positive when you have the disease' does not tell you that you have the disease when the test is positive.

WORKED FIRST: do the whole thing in numbers

Question: is 'if a then b' true when a is false and b is false?

Step 1. Throw the words away. a is false, so **a = 0**. b is false, so **b = 0**.

Step 2. Write the conditional as arithmetic. a -> b is max(1 - a, b).

Step 3. Substitute. max(1 - 0, 0) = max(1, 0) = **1**.

Step 4. Read it back. 1 means true.

Cross-check with the other reading. a -> b is 1 exactly when a is no bigger than b. Is 0 <= 0? Yes. Same answer, and it took two seconds.

Notice what never entered the calculation. What a and b were about. The machine had two numbers and produced a third. There was nowhere for a moon to go.

For the questions below, convert to 0 and 1 first, on their own line. Most confusion in this part comes from arguing with the English while the arithmetic sits there settled.

WORKED FIRST: is 'if a then b' the same as 'if b then a'?

Step 1. Do not argue about it. Build both tables.

Step 2. Table for 'if a then b'. False only when a is T and b is F. Rows: T,T gives T. T,F gives **F**. F,T gives T. F,F gives T.

Step 3. Table for 'if b then a'. False only when b is T and a is F. Rows: T,T gives T. T,F gives T. F,T gives **F**. F,F gives T.

Step 4. Compare row by row. Row two: first says F, second says T. **They differ.**

Answer: no, and row two is the counterexample. That row is a world where a is true, b is false: it rained but the ground is dry, under a canopy.

Notice what settled it. Not an argument. One row of a table you wrote down. That is the entire method of this part of the book.

Now build both tables in your own book for the questions below. Four rows each, side by side, and compare downwards.

PRACTICE

1.0a Work these out as arithmetic: (a) 1 - 0 (b) min(1, 0) (c) max(0, 0) (d) 1 x 0. Say which connective each one is.

1.0b Work out max(1 - a, b) for all four combinations of a and b being 0 or 1. Which row gives 0?

1.0c 'If grass is purple then fish can knit.' Convert both parts to 0 or 1, then compute. Is the sentence true?

1.0d In one sentence, say what the machine would need in order to notice that grass and fish are unrelated.

1.1 Build the table for NOT a. Two rows.

1.2 Build the table for a AND (NOT a). What do you notice about the answer column?

1.3 Build the table for a OR (NOT a). Same question.

1.4 Build the table for NOT (a OR b) and for (NOT a) AND (NOT b). Do the columns match?

1.5 Write the contrapositive of: if the alarm sounds, there is smoke.

1.6 Write the converse of the same sentence. Give a situation where the original is true and the converse is false.

1.7 Is 'if pigs fly then I am the Queen' true or false? Say why in one sentence.

Valid is not the same as true

This is the most useful distinction in the book and it takes one page.

An **argument** is premises plus a conclusion. It is **valid** when the conclusion cannot be false while the premises are true.

Notice what that does not say. It says nothing about whether the premises are actually true. **Validity is about the wiring, not about the world.**

argument	premises	verdict	
all cats are mammals / Tibbles is a cat / so Tibbles is a mammal	true	**valid**	the good case
all cats are green / Tibbles is a cat / so Tibbles is green	false	**valid**	wiring fine, input wrong
all cats are mammals / Tibbles is a mammal / so Tibbles is a cat	true	**invalid**	input fine, wiring wrong. Tibbles could be a dog
the moon is cheese / so grass is green	mixed	**invalid**	conclusion true, argument worthless

All four combinations happen. A valid argument with false premises tells you nothing about the world. An invalid argument can land on a true conclusion by luck. **The two things are independent**, and almost every bad public argument confuses them.

Logic checks the pipe, not the water. A perfect pipe carrying sewage delivers sewage, reliably and on time.

The two rules that work, and the two that do not

Modus ponens	if a then b / a / so b	**valid**	the promise held and the condition happened
Modus tollens	if a then b / not b / so not a	**valid**	the promise held and the outcome did not happen, so the condition cannot have
affirming the consequent	if a then b / b / so a	**invalid**	b could have another cause. This is the converse trap wearing a suit
denying the antecedent	if a then b / not a / so not b	**invalid**	the inverse trap. Nothing was promised about the a-is-false case

Check the invalid ones with a row. Take 'if it rains the ground is wet'. The pipe burst: the ground is wet and it did not rain. That single row kills affirming the consequent, and you do not need anything else.

WORKED FIRST: test an argument in four steps

Argument: If the shop is open, the light is on. The light is on. So the shop is open.

Step 1. Name the parts. a is 'the shop is open'. b is 'the light is on'.

Step 2. Write the shape. if a then b / b / therefore a.

Step 3. Hunt for one row where the premises are true and the conclusion is false. That means: a is F, b is T.

check premise 1: if F then T, which is **T**. Fine.

check premise 2: b is **T**. Fine.

check conclusion: a is **F**. So the conclusion fails while both premises hold.

Step 4. Verdict: invalid, and the row IS the counterexample. The shop is shut and somebody left the light on.

You only ever need one row. To show an argument invalid, find the row. To show it valid, check that no such row exists.

Set the questions below out the same way in your own book: name the parts, write the shape, hunt for the bad row.

PRACTICE

2.1 Name the shape and give the verdict: if it is Tuesday the bins go out / it is Tuesday / so the bins go out.

2.2 Same for: if it is Tuesday the bins go out / the bins went out / so it is Tuesday. Find the bad row.

2.3 Same for: if you revise you pass / you did not revise / so you did not pass. Find the bad row.

2.4 Same for: if you revise you pass / you did not pass / so you did not revise.

2.5 Build a valid argument with false premises and a false conclusion.

2.6 Build an invalid argument whose conclusion happens to be true.

ALL and SOME

Everything so far treats a claim as a single lump. To say something about a collection you need two more words, and they are the last two pieces of classical logic.

ALL	for every one of them	written with an upside-down A
SOME	for at least one of them	written with a backwards E

Remember these two things about them, because they cause most of the trouble:

SOME means at least one	It does not mean 'not all'. If all seven cats are black, then some cats are black is still true.
ALL over nothing is true	'All the unicorns in my garden are purple' is true, because there are none to be otherwise. This is vacuous truth again, and it is the same hazard from three pages ago.

Negating them: the swap

This is a rule you can derive rather than memorise.

```
NOT (all are P) is SOME are not P

NOT (some are P) is ALL are not P
```

Why. To break 'all the eggs are fresh' you need one bad egg, not a whole bad box. To break 'some egg is fresh' every single one has to be off.

It is De Morgan again, at a larger scale. ALL is a long AND, SOME is a long OR, and the negation swaps them exactly as before. **One rule, appearing twice.**

Order matters, and it matters a great deal

Everyone has a mother.	for ALL people, SOME woman is their mother	true, and each person has their own
Some woman is everyone's mother.	SOME woman is such that for ALL people she is their mother	false, and it is a different claim

Same words, different order, different meaning. Swapping the order of two different quantifiers is a real error with a real name, and it is worth checking for whenever a claim has both.

WORKED FIRST: negate a quantified sentence, in steps

Sentence: All the students passed every exam.

Step 1. Write the shape. ALL students, ALL exams, passed.

Step 2. Push the NOT inwards one quantifier at a time.

NOT(ALL students ...) becomes SOME student such that NOT(...)

NOT(ALL exams passed) becomes SOME exam not passed

Step 3. Put it together. SOME student, SOME exam, did not pass.

Step 4. Say it in English. At least one student failed at least one exam.

Check it is the weakest thing that breaks the original. One student failing one exam is enough. You do not need everyone to fail everything, and if you wrote that, you negated too hard.

Every ALL becomes a SOME and every SOME becomes an ALL. Do them one at a time, left to right, and write each stage on its own line.

PRACTICE

3.1 Negate: all swans are white.

3.2 Negate: some meetings are useful.

3.3 Negate: every door in the building is locked.

3.4 If there are no fish in the tank, is 'all the fish in the tank are goldfish' true or false? Say why.

3.5 Give a case where 'some' is true and 'all' is also true.

3.6 Explain the difference between 'everyone has a favourite song' and 'some song is everyone's favourite'.

What the two-value machine cannot do

Every machine has a place it stops, and this one has four. None of these is a defect. They are the price of the settings.

HOW IT BREAKS

It has nowhere to put 'unknown'. Every claim must be T or F. 'The number of grains in that heap is even' is one or the other and you do not know which, and the machine has no way to represent your not knowing. Part Two adds a value for this.

It cannot handle vague edges. Is a man with 900 hairs bald? The machine demands a yes or no, and wherever you draw the line, one hair either side gives the opposite answer. That is the sorites problem and it is old.

It treats any contradiction as total collapse. From 'a and not a' the machine will derive absolutely anything, including that the moon is cheese. This is called explosion, and it means one bad record in a large database makes every conclusion available. Part Two has a machine that survives this.

It cannot say 'might'. 'It is raining' and 'it might be raining' are different claims and this machine has one slot. Part Three is entirely about that gap.

Explosion, shown

Suppose you accept both a and NOT a. Here is how the machine gets from there to anything at all, in three steps, using only rules from this part.

```
1. a (given)
2. a OR moon-is-cheese (from 1: if a is true, 'a or X' is true)
3. NOT a (given)
4. moon-is-cheese (from 2 and 3: one side failed, so the other must hold)
```

Every step is legitimate. The conclusion is absurd. **The machine is working correctly and the output is useless**, which tells you something about the settings rather than about the moon.

This is why the two-value machine is unusable on any real body of information. Every large medical record, legal code or engineering archive contains contradictions, and a machine that yields everything from one contradiction yields nothing.

PART TWO

TURNING THE V DIAL

Add a third value and see what happens. Everything in this part was computed, and you can rebuild any of it from the definitions.

The four things a third value can mean

Write the third value as **?**. Before choosing any tables, notice that people mean at least four different things by it.

gap	neither true nor false	the claim has no truth value yet. 'The next king of France is bald' with no France
glut	both true and false	two records disagree and you have to keep both
unknown	one of them, but I do not know which	the coin has landed under the cup
meaningless	the question does not apply	'is Tuesday heavier than green'

These give different machines. That is the finding of this part. Choosing what ? means is choosing which tables you get, and people who do not state which one they mean produce arguments that cannot be checked.

Theta: which values you may assert

Here is the setting almost nobody mentions and it decides more than the tables do. This is **theta** from Part Zero, turned on the value space.

Theta is the cut that says which values are good enough to assert. In the two-value machine the cut sits above F and there was nothing to notice, because there was only one place to put it.

With three values there are two places to put it, and the choice is real:

cut above ?	only fully true counts	assertible: {T}	gives **K3** and **■3**
cut above F	the middle counts too	assertible: {?, T}	gives **LP**

Same values. Same tables. Different theta. Different logic. Watch what it does.

The standard textbooks call the assertible set the **designated set** and write it D. That is the same thing under a different name, and it is worth knowing both because you will meet D everywhere else.

K3: strong Kleene, the gap machine

Read ? as **no value yet**. Then work out each table by asking: if the missing one were filled in either way, would the answer change?

AND. If either side is F, the answer is F whatever the other is, so F AND ? is F. But T AND ? depends on what ? turns out to be, so it stays ?.

AND	F	?	T
F	F	F	F
?	F	?	?
T	F	?	T

OR. Mirror image. If either side is T the answer is T, so T OR ? is T. F OR ? is undecided.

OR	F	?	T
F	F	?	T
?	?	?	T
T	T	T	T

NOT. The opposite of unknown is unknown.

a	NOT a
F	T
?	?
T	F

What breaks, and it is exactly one thing

Computed across all fifteen standard laws: **K3 keeps fourteen and loses one**.

`LEM: a OR (NOT a) is not always designated`

Check the row. If a is ?, then NOT a is ?, and ? OR ? is ?. And ? is not in D, so the law fails.

This is correct behaviour, not a defect. If a claim genuinely has no truth value, then 'it or its opposite' should not be assertible either. K3 is the machine that refuses to guess.

LP: the glut machine, and surviving a contradiction

Same three values. Same tables. One change: theta cuts lower, so the assertible set is {?, T}. Read ? as both true and false at once.

Now watch what falls out. Computed across the same fifteen laws, LP keeps twelve and loses three:

LNC	a AND NOT a is never assertible	**fails**: if a is ?, then a AND NOT a is ?, which IS in D
NoGlut	nothing is both	**fails by construction**: that is what ? means here
MP	from 'if a then b' and a, conclude b	**fails**: a is ?, b is F. The conditional comes out ?, which is assertible, and a is assertible, but b is F

WHAT LOSING MODUS PONENS BUYS YOU

Losing modus ponens sounds catastrophic. Look at what it prevents.

In LP, **explosion does not happen**. You can accept a and NOT a and still not derive that the moon is cheese.

Run the explosion argument from Part One with a set to ?. Step 4 was 'one side failed, so the other must hold'. In LP the first side did not fail; it is ?, which is assertible. The inference does not fire.

So the contradiction stays local. One bad record poisons the claims that touch it and nothing else.

That is the trade, stated plainly: **LP gives up a rule of inference to gain the ability to hold inconsistent information without going mad**. For a medical database or a legal code, that is very often the right purchase.

■3: the same values, different connectives

■ukasiewicz built a three-valued logic in 1920 to handle statements about the future. It comes in two versions and **they disagree with each other**, which is the most instructive fact in this part.

Version one uses min and max, exactly like K3, and differs only in the conditional. Version two uses a different pair of connectives.

AND-2	F	?	T
F	F	F	F
?	F	F	?
T	F	?	T

Look at the middle cell. **? AND ? gives F**, not ?. So a claim combined with itself does not give itself back.

Computed: this version keeps eleven of fifteen and loses idempotence for both AND and OR, plus distribution and absorption. The first version keeps fourteen and loses only LEM, exactly like K3.

Two systems, both correctly called ■ukasiewicz three-valued logic, giving opposite answers on four laws. So the question 'does law X hold in ■3' has no answer until you say which connectives you mean. The name does not determine the machine.

This matters beyond the example. Whenever somebody tells you what a named logic does, the honest response is: **which presentation?** A name is a label on a box, and two boxes can carry the same label.

B3: the infectious middle

One more, and it is the simplest to state. Weak Kleene says: if anything in the expression is ?, the whole expression is ?. No exceptions.

AND	F	?	T
F	F	?	F
?	?	?	?
T	F	?	T

Compare with K3, where F AND ? was F. Here it is **?**. The middle value spreads through everything it touches, which is why it is called infectious.

This models 'the question is meaningless' rather than 'the answer is unknown'. If part of your sentence is nonsense, the sentence is nonsense, and no amount of falsity elsewhere rescues it. It is also exactly how a spreadsheet handles an error cell, and how a database handles NULL.

Computed: thirteen of fifteen, losing LEM and absorption.

The scoreboard

Every machine so far, computed against the same fifteen laws.

machine	V	assertible (theta)	connectives	keeps	loses
BOOL	T F	{T}	min / max	**15**	nothing
K3	T ? F	{T}	min / max	14	LEM
■3-lattice	T ? F	{T}	min / max	14	LEM
B3	T ? F	{T}	infectious	13	LEM, absorption
LP	T ? F	{?, T}	min / max	12	LNC, no-glut, modus ponens
■3-strong	T ? F	{T}	odot / oplus	11	idempotence x2, distribution, absorption

THE SACRIFICE THEOREM

Take every possible way to wire a three-valued logic that behaves classically at the corners. Three choices for NOT, 243 for AND, 243 for OR, and two choices of D.

That is **354,294 machines**, and somebody enumerated all of them against the fifteen laws.

Not one gets all fifteen. Not one. You may have excluded middle or idempotence, De Morgan or modus ponens, but the full set is not available at any price.

The result was then re-run under three different definitions of non-contradiction, 1,062,882 checks in total, and it survived all three. It does not depend on how you define the law.

So there is no best three-valued logic. There is a menu, and every item costs something. Choosing one is an engineering decision about what you can afford to lose.

How to choose

you have missing data	**K3**	refuses to guess. Loses excluded middle, which you did not need
you have conflicting data	**LP**	survives contradiction. Loses modus ponens, which hurts
you have meaningless entries	**B3**	errors propagate rather than vanish. This is what your spreadsheet does
you have complete, clean data	**BOOL**	keeps everything. Use it whenever you honestly can

Classical logic is not wrong. It is the machine you get when you assume complete, consistent, meaningful information, and when you actually have that, it is the best machine available. The other four exist because you often do not.

WORKED FIRST: work out a K3 truth value in steps

Question: in K3, what is (a OR NOT a) AND b, when a is ? and b is T?

Step 1. Work from the inside out and write each stage.

Step 2. NOT a, where a is ?. Look it up: NOT ? is **?**.

Step 3. a OR NOT a, which is ? OR ?. Look it up in the OR table: row ?, column ? gives **?**.

Step 4. ? AND b, where b is T. Look it up in the AND table: row ?, column T gives **?**.

Answer: ?

Step 5. Is it assertible? In K3 theta cuts above ?, so ? is not assertible. So **no**.

Now change one setting and redo it. In LP, everything above is identical until the last line: theta cuts lower, so the same value ? IS assertible, and the answer flips. **Nothing about the tables changed. Only theta did.**

For the questions below, write each stage on its own line, with the table you looked it up in. Then check D at the end, separately.

PRACTICE

4.1 In K3, what is ? AND F? What is ? OR T?

4.2 In K3, work out NOT(? AND T) and (NOT ?) OR (NOT T). Do they match?

4.3 In K3, is a OR NOT a assertible when a is ?? Show each step.

4.4 In LP, same question. What changed?

4.5 In B3, what is F AND ?? Why does it differ from K3?

4.6 A hospital database has a record saying a patient is allergic to penicillin and another saying they are not. Which machine would you run the alerts on, and what have you given up?

PART THREE

THE GRAPH MACHINE

Modal logic. It has a reputation for being the mystical end of the subject. It is a network with arrows, and two words meaning ALL and SOME.

The same three dials, turned somewhere else

Modal logic is usually taught as a new subject. It is the machine from Part Zero with the dials pointed at a different thing.

V	the value space	no longer truth values. Now it is **situations, and the arrows between them**. Dots and lines.
G	the gradient	no longer AND and OR. Now it is **stepping along an arrow**: box means every step, diamond means some step.
theta	the threshold	**the constraint on the arrows.** Demand that every dot sees itself, or that two steps is also one step. This is what names the system.

That third row is the whole of Part Three. **K, T, S4 and S5 are not four logics.** They are one machine with theta tightened by degrees, and each tightening buys an axiom.

So there was never a fourth dial. Theta cut on the values gave you K3 and LP in Part Two. Theta cut on the arrows gives you T and S4 and S5 here. Same setting, different surface.

The whole dictionary

Every intimidating word in modal logic has a boring engineering equivalent. Here is the entire translation, and there is nothing else to learn.

possible world	**a node**	a dot on a diagram
accessibility relation	**the arrows**	which dots point to which
frame	**the graph**	the dots and arrows together
model	**the graph, plus labels**	which dots are marked true
necessarily p, written []p	**ALL the dots I point to are marked**	a universal, with a range
possibly p, written <>p	**SOME dot I point to is marked**	an existential, with a range
valid on a frame	**true at every dot, under every labelling**	check all the cases

'Possible world' is the phrase doing the damage. It sounds like a claim that other universes exist. In the machine it is a dot. Nobody thinks the rows of a truth table are places, and this is the same object with worse marketing.

And box and diamond are just ALL and SOME from Part One, with the range restricted to **wherever your arrows go**. That is the only new idea in this entire part.

What a dot is, in practice

It depends entirely on what you are modelling, and it is never mystical.

a computer program	a state the program can be in
a game	a position on the board
what somebody knows	a scenario consistent with what they have seen
what is permitted	a way things could go without breaking a rule
time	a moment

The question that is not a question

Is p possible? That is not a complete question. Possible **from where**?

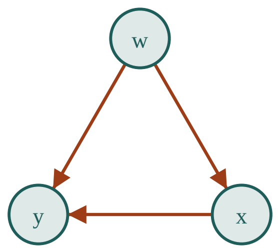
Take a chess position. Is it possible for the bishop to be one square to the left? Relative to the rules of chess: no. Relative to physics: obviously, pick it up and move it.

Same board, same piece, opposite answers, because the arrows changed.

The arrows are the answer to 'possible relative to what'. They are an input you choose, not something you discover, and almost every confusing modal argument is confusing because nobody said which arrows they meant.

Reading a graph

Here is a graph with three dots. An arrow from w to x means: standing at w, x is one of the situations you are considering.



a one-way chain

Standing at **w**, you point to x and y. So:

[]p at w	is p marked at **both** x and y?
<>p at w	is p marked at **either** x or y?

Notice that w does **not** point to itself. So whether p is marked at w is irrelevant to []p at w. That will matter enormously in a moment.

WORKED FIRST: evaluate a box and a diamond on a graph

Setup. Three dots w, x, y. Arrows: w to x, w to y, x to y. Mark p as true at **y only**.

Question 1: is []p true at w?

Step 1. List where w points: x and y.

Step 2. Box means ALL of them. Is p marked at x? **No.**

Step 3. One failure is enough. **[]p is false at w.**

Question 2: is <>p true at w?

Step 1. Same list: x and y.

Step 2. Diamond means SOME. Is p marked at y? **Yes.**

Step 3. One success is enough. **<>p is true at w.**

Question 3: is []p true at y?

Step 1. List where y points: **nowhere**.

Step 2. ALL of nothing. There is no failure available, so it holds. **[]p is true at y**, and so is []q for anything else you like.

That last one is vacuous truth again, from Part One, arriving in a new costume. A dot with no arrows out satisfies every box statement there is.

For the questions below, always write the list of where the dot points first. That one line removes almost every mistake in this part.

The correspondence: axioms are settings of theta

Here is the result that makes modal logic an engineering subject. There is an **exact** match between properties of the graph and axioms of the logic. Not a rough alignment. An exactly-when.

Each of these was checked against **every possible arrangement of arrows on three dots**, all 512 of them, under every labelling. Each correspondence is exact in both directions.

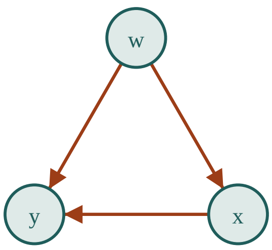
axiom	schema	shape	in English
T	[]p -> p	reflexive	every dot has a self-loop
D	[]p -> <>p	serial	every dot has at least one arrow out
4	[]p -> [][]p	transitive	if you can get there in two steps, you can get there in one
B	p -> []<>p	symmetric	every arrow goes both ways
5	<>p -> []<>p	euclidean	everything I see, sees everything I see

Read that table as build instructions and the axioms stop being principles you assent to. **Set theta and you get the axiom free.**

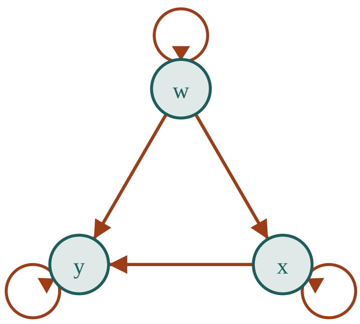
The left column is what the textbooks give you as a list to memorise. The third column is what you actually choose. Nobody picks axioms; they pick a constraint on the arrows, and the axioms are the receipt.

T, and why chess breaks it

T says: if p is necessary then p is true. It holds exactly when every dot has a self-loop, which means the situation you are actually in counts among the situations you are considering.



no self-loops: T fails



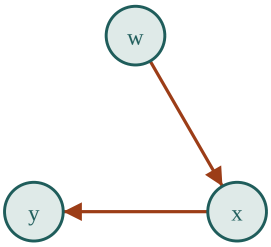
self-loops: T holds

Chess is the clean counterexample. From a given position, the positions you can reach in one legal move do **not** include the one you are already in, because you cannot pass.

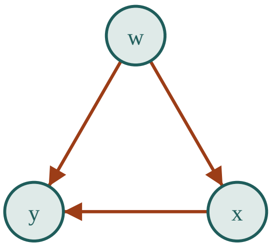
So chess is not reflexive, and T fails: **every position I can move to has feature X** tells you nothing about whether the current position has X. Stated in chess that is obvious. Stated as an axiom it looks like a philosophical commitment.

4, and stacking boxes

4 says: if p is necessary, then it is necessary that p is necessary. It holds exactly when the graph is transitive.



not transitive: 4 fails

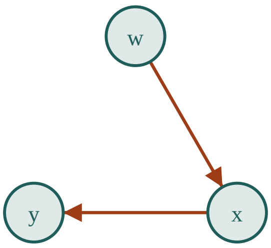


transitive: 4 holds

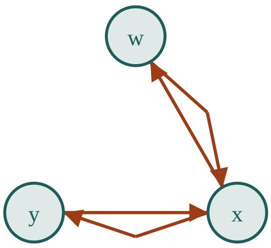
In the left graph, w points to x and x points to y, but w does not point to y. So standing at w you cannot see y, and something you would say about 'everything reachable' misses it. Add the shortcut and the axiom appears.

B and 5, and seeing back

B says: if p is true, then necessarily it is possible. It needs every arrow to go both ways.



one-way: B fails



two-way: B holds

5 says: if p is possible, then necessarily it is possible. It needs the euclidean property, which is the strongest condition on the list and is worth reading slowly: anything I can see must be visible from everything else I can see.

The named systems

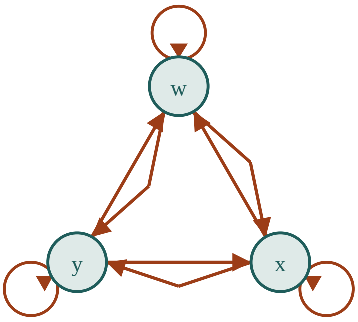
Stack the constraints and you get the systems with names. There is nothing more to them than **how tight you set theta**.

system	theta: the constraint on arrows	axioms you get	frames that qualify
K	no conditions at all	nothing beyond the basic machinery	512 of 512
D	serial	D	343 of 512
T	reflexive	T, D	64 of 512
B	reflexive + symmetric	T, D, B	8 of 512
S4	reflexive + transitive	T, D, 4	29 of 512
S5	reflexive + symmetric + transitive	T, D, 4, B, 5	**5 of 512**

Look at the last column. Of the 512 possible three-dot networks, **five** are S5 frames. Just under one percent. Keep that number; Part Five needs it.

S5 drawn

S5 requires the arrows to form an equivalence: everything in a cluster points to everything in that cluster, including itself.



S5: everyone sees everyone

The consequence is that **possibility does not vary by position**. Every dot in the cluster sees the same set, so every dot agrees about what is possible. Possibility becomes absolute.

A detail that surprises people: S5 does **not** require the graph to be connected. Two separate clusters, with no path between them, is still S5, because the condition is satisfied inside each. Possibility is absolute **within a component**, and S5 says nothing across the gap.

The flavours

The same graph machinery gets read several different ways depending on what the dots and arrows are. The mathematics does not change. What changes is which system is honest.

alethic	necessarily / possibly	ways things could be	usually S5, and Part Five is about why that is a large claim
epistemic	I know / for all I know	scenarios consistent with my evidence	T is fair: what you know is true. 5 is not
doxastic	I believe	scenarios consistent with my beliefs	**drop T**: beliefs can be false. Usually D
deontic	obliged / permitted	ways things could go without breaking a rule	**drop T**: obligations get violated. D only
temporal	always / sometime	moments in time	transitive, and usually not symmetric
provability	provable in a system	what a formal system can reach	**drop T**, and this one is genuinely strange

Deontic logic is the one that shows why T matters. **T says if p is necessary then p is true.** In a deontic reading that says: if something is obligatory then it happens. Obviously false. People break rules constantly.

So deontic logic drops T and keeps D, which only says the obligations are not self-contradictory. That is a much weaker claim and it is the honest one.

Which axioms you may claim is decided by what your dots and arrows actually are. Copying S5 across from metaphysics into epistemics because it is tidy is how most bad modal arguments start.

WORKED FIRST: read the logic off a graph, in steps

Graph: three dots. Arrows: w to x, x to y, w to y, and a self-loop on every dot.

Step 1. Reflexive? Does every dot have a self-loop? **Yes.** So you get **T**.

Step 2. Serial? Does every dot have at least one arrow out? Yes, the self-loops guarantee it. So you get **D**. Note that reflexive always gives serial free.

Step 3. Transitive? w to x and x to y: is there w to y? **Yes.** Check every such pair. So you get **4**.

Step 4. Symmetric? Is there an arrow x to w? **No.** So B fails.

Step 5. Euclidean? w sees x and y. Does x see y? Yes. Does y see x? **No.** So 5 fails.

Verdict: T, D and 4. That is S4.

You never mentioned an axiom until the end. You looked at arrows and read the answer off. That is the whole method.

For the questions below, draw the graph first, then run the five checks in this order, writing yes or no beside each.

PRACTICE

5.1 Draw three dots with arrows w to x, x to y, y to w, and no self-loops. Which of the five shapes does it have?

5.2 Add self-loops to all three. Which shapes now?

5.3 Draw the graph where every dot points to every dot. Run all five checks. Which system is it?

5.4 Take the graph in 5.1 and mark p true at y only. Is []p true at w? Is <>p true at w? Show the list of where w points first.

5.5 A dot with no arrows out at all. Is []p true there? Is <>p? Explain both.

5.6 For chess, is the accessibility relation reflexive? Say why in one sentence, and say which axiom that costs you.

5.7 Should belief be modelled with T? Give a reason.

PART FOUR

THE BREAKS

What these machines cannot tell you, and the specific mistakes people make when they forget that.

Coherence is not authority

Here is the error this part exists for, stated once and then examined.

A logic being consistent inside itself says nothing whatever about whether the world is shaped like it.

You saw this in Part One under a different name. An argument can be perfectly valid and tell you nothing, because validity is about the wiring and not about the water. A whole logical system is the same thing scaled up: **internal coherence is a property of the machine, and it is free**. You can build a consistent system saying almost anything.

Euclidean geometry	perfectly coherent	space is not Euclidean
Newtonian mechanics	perfectly coherent	not how things move at speed
a hyperbolic geometry	perfectly coherent	describes some surfaces and not a flat table
S5 modal logic	perfectly coherent	requires something in Part Five worth looking at

Every one of those is a working machine. None of them earns the right to tell you how the world is by being tidy. **That has to be settled outside the system, and no amount of internal work substitutes for it.**

A system's consistency is a receipt saying the parts do not contradict each other. It is not a receipt saying the parts describe anything.

The S5 problem

S5 is the strongest of the standard systems and by far the most used in metaphysical argument. It is worth being precise about what adopting it commits you to, because it is a great deal more than it looks.

What S5 actually demands

S5 requires the arrows to be reflexive, symmetric and transitive: everything in a cluster sees everything in that cluster.

Now read that in the language it is usually deployed in, where dots are 'ways things could be'.

WHAT YOU ARE CLAIMING WHEN YOU ADOPT S5

Every possibility is visible from every possibility. There is no corner of the space of ways things could be that is hidden from any other corner.

Possibility does not vary by position. What is possible from here is exactly what is possible from anywhere, so nothing about your situation restricts what you can consider.

So possibility is absolute, and the modal facts are the same everywhere and cannot be discovered, only stipulated.

Put plainly: **adopting S5 is claiming complete access to the entire space of ways things could be, from where you are standing, right now.**

That is a claim of total knowledge about a space you have, by construction, never surveyed. It is not a modest technical assumption. It is the largest premise in most arguments that contain it.

The number from Part Three

Of the 512 possible three-dot networks, **five** satisfy S5. Just under one percent.

That number does not prove anything about metaphysics. It does give a sense of proportion: S5 is not the natural or default shape of a network. It is a very tightly constrained special case, and something has to justify demanding it.

Where S5 is completely fine

This is not an argument that S5 is bad. It is a superb tool where the space really is closed and surveyable.

a finite state machine	you have the full state list. Every state is enumerable and you own the specification
a game with fixed rules	the legal positions are defined by the rules and there is nothing outside them
a closed database	the records are the records
a mathematical structure you defined	you built it, so you know its extent

In every one of those, the space is **something you made or can list in full**. Absolute possibility is fair because the space is finite and yours.

Where it is not

The trouble starts when the same system is applied to a space nobody made and nobody has surveyed: the ways reality could have been.

There, S5 asserts complete access to a space of unknown extent. And the assertion cannot be checked, because checking it would require the survey that S5 is standing in for.

The step that carries the weight

Almost every argument that needs S5 needs it for one specific move:

```
<>[]p -> []p
```

If it is possible that p is necessary, then p is necessary.

Read it as graph reachability and it is transparent. It says: if somewhere you can reach thinks p holds everywhere IT can see, then p holds everywhere **you** can see. That is only guaranteed if you and that dot see the same set.

Computed over all 512 frames:

S5	reflexive + symmetric + transitive	**VALID**
B	reflexive + symmetric	fails
S4	reflexive + transitive	fails
T	reflexive	fails
K	anything at all	fails

It fails in B and it fails in S4. You need symmetry **and** transitivity together. That is a one-line difference in a graph and an enormous difference in what you may conclude.

When somebody's argument uses that step, they have not made a small technical choice. They have assumed the shape of the whole space, and the assumption belongs in the premise list where everyone can see it.

Reification: treating an output as an input

The second error, and it is more common than the first because it is harder to see.

Here is the shape of it. A modal machine takes inputs: dots, arrows, labels. It produces outputs: which formulas hold where. **The error is taking an output and treating it as a thing the machine discovered**, then using it as an input somewhere else.

input	the dots, the arrows, the labels	you chose these
output	[]p holds at w	the machine computed this
the error	'so necessity is a real feature of p'	the output has been promoted to a discovery

Why it is circular

Walk it round. You put in a graph. The graph decides what comes out. You then treat what came out as evidence about the world, and use it to justify the graph.

```
choose arrows -> compute []p -> cite []p as a fact ->
-> justify the arrows -> choose arrows ...
```

Nothing entered that loop from outside. The conclusion was in the arrows before the machine ran, and the machine's job was to make it explicit, not to establish it.

A modal output is a receipt for the graph you supplied. Reading it as a discovery about reality is reading the receipt as though it were the shopping.

The bare particular, and the boundary nobody paid for

The third error is the deepest and it hides inside the word **possible** itself.

Consider a claim of the form: **there is a possible situation containing an object with exactly these properties and no others.** That move appears constantly in modal argument.

Ask what it cost to make that object exist as one thing.

EVERY OBJECT NEEDS A BOUNDARY, AND BOUNDARIES ARE NOT FREE

To have an object at all, something must hold it apart from everything else. A boundary. Where it stops and the rest begins.

In the real world boundaries are **maintained**, and the maintenance is visible. A cell has a membrane and pumps ions to keep it. A country has borders and pays for them. A file has a header. Two puddles that touch stop being two puddles.

A bare particular is an object posited with a boundary and no account of what maintains it. It is asserted to be one thing, distinct from everything else, at no cost, in a space nobody surveyed.

The modal machinery lets you do this because the dots are given as inputs. The graph never asks what makes a dot a dot.

So the boundary was smuggled in as a free assumption, and the argument then draws conclusions from the object's distinctness. The distinctness was assumed, not established.

This is not an objection to modal logic. The machine is doing exactly what it says: it takes dots as given. **The objection is to reading the machine's willingness to accept a dot as evidence that the dot could exist.**

A graph will happily accept a node labelled 'a round square'. It will compute perfectly correct answers about it. That the arithmetic works tells you nothing about whether the node describes anything.

How to read a modal argument

Four questions. They dissolve most of what you will meet, and none of them requires you to know any more logic than Part Three.

1	**Possible relative to what?** Name the arrows. If the argument never says, that is your first finding and usually your last.
2	**Which system, and what does it cost?** If a step goes from <>[]p to []p, it needs S5, which is a claim of complete access to the whole space. Put it in the premise list.
3	**Did the graph change halfway through?** This is where nearly everything goes wrong. A premise is made plausible on the cheap reading of possible, meaning 'for all I know', and then a conclusion is drawn on the expensive one, meaning 'there is genuinely a dot there'. Different graphs. The symbols do not record which you meant.
4	**What is holding the objects apart?** If a possible situation contains an object, ask what maintains its boundary. If the answer is nothing, the distinctness was assumed rather than shown.

The cheapest and most common error is question three. Watch for it: 'it is possible that X' meaning 'I cannot rule it out', followed three lines later by 'so there is a world where X', meaning something far stronger. Those are two graphs.

HOW IT BREAKS

None of this shows modal logic is wrong. The machinery is correct and it is genuinely useful. Model checkers verify aircraft software with exactly these tools, and they work.

The difference is where the graph comes from. In a program, the arrows are the transitions and they are in the source code. Nobody argues about them. In metaphysics the arrows are supplied by intuition, and that is the same machine on a very different input.

So modal logic is at its strongest where the graph is given by the subject matter, and at its most contested where the graph has to be guessed. Same tool. The whole difference is the cost of the input.

PART FIVE

THE OPERATOR ATLAS

Every connective in every machine, in one place. All computed. Use it as a reference, not as reading.

NOT, in every machine

machine	NOT F	NOT ?	NOT T	note
BOOL	T	F	-	-
K3, LP, ■3, B3	T	?	F	the middle is its own opposite

Every three-valued machine in this book uses the same NOT. The differences are all in AND, OR and the conditional.

AND, in every machine

Rows are the left input, columns the right.

BOOL

AND	F	T
F	F	F
T	F	T

the base case, for comparison

K3 / LP / ■3-lattice

AND	F	?	T
F	F	F	F
?	F	?	?
T	F	?	T

minimum. F wins over ?, because F AND anything is F

■3-strong

AND	F	?	T
F	F	F	F
?	F	F	?
T	F	?	T

note the middle cell: **? AND ? gives F**, so idempotence fails

B3 (weak Kleene)

AND	F	?	T
F	F	?	F
?	?	?	?
T	F	?	T

infectious: **F AND ? gives ?**, not F. The middle spreads

OR, in every machine

BOOL

OR	F	T
F	F	T
T	T	T

the base case

K3 / LP / ■3-lattice

OR	F	?	T
F	F	?	T
?	?	?	T
T	T	T	T

maximum. T wins over ?, because T OR anything is T

■3-strong

OR	F	?	T
F	F	?	T
?	?	T	T
T	T	T	T

? OR ? gives T, so idempotence fails here too

B3 (weak Kleene)

OR	F	?	T
F	F	?	T
?	?	?	?
T	T	?	T

infectious again: **T OR ? gives ?**

The conditional, in every machine

This is where the machines differ most, and where the name of a logic least determines its behaviour.

BOOL

a -> b	F	T
F	T	T
T	F	T

false only on T then F

K3 / LP (material)

a -> b	F	?	T
F	T	T	T
?	?	?	T
T	F	?	T

built as NOT a OR b. Note **? then ? gives ?**, so 'if p then p' is not assertible

■3 (both versions)

a -> b	F	?	T
F	T	T	T
?	?	T	T
T	F	?	T

? then ? gives T. ■ukasiewicz built the conditional so that 'if p then p' always holds

Look at the middle cell of the last two tables. Under the material conditional, **if p then p** is not assertible when p is ?. Under the ■ukasiewicz conditional it is.

That is a genuine separator, and unlike excluded middle it survives whichever set of connectives you pick. If you want one fact to distinguish these systems, use this one rather than LEM.

The fifteen laws, machine by machine

law	says	BOOL	K3	LP	■3-latt	■3-str	B3
LNC	a AND NOT a is never assertible	yes	yes	**no**	yes	yes	yes
LEM	a OR NOT a is always assertible	yes	**no**	yes	**no**	yes	**no**
DN	NOT NOT a is a	yes	yes	yes	yes	yes	yes
NoGlut	nothing is both	yes	yes	**no**	yes	yes	yes
MP	modus ponens	yes	yes	**no**	yes	yes	yes
ANDidem	a AND a is a	yes	yes	yes	yes	**no**	yes
ORidem	a OR a is a	yes	yes	yes	yes	**no**	yes
DeMorgan	both directions	yes	yes	yes	yes	yes	yes

law	says	BOOL	K3	LP	■3-latt	■3-str	B3
Distrib	AND over OR	yes	yes	yes	yes	**no**	yes
Absorb	absorption	yes	yes	yes	yes	**no**	**no**

Commutativity and associativity of AND and OR hold in all six and are left out of the table to keep it readable. That is the fifteen.

Modal operators, machine by machine

[]p	ALL dots I point to are marked	same in every system
<>p	SOME dot I point to is marked	same in every system
[]p	equals NOT <> NOT p	the ALL and SOME swap from Part One
<>p	equals NOT [] NOT p	same rule, other direction

The operators never change. Box is always ALL and diamond is always SOME. What changes between K, T, S4 and S5 is **the graph**, and therefore which formulas come out true. That is worth saying plainly because the systems are usually presented as different logics rather than as the same logic on different networks.

Which system gives which axioms

system	T	D	4	B	5	frames of 512
K	-	-	-	-	-	512
D	-	yes	-	-	-	343
T	yes	yes	-	-	-	64
B	yes	yes	-	yes	-	8
S4	yes	yes	yes	-	-	29
S5	yes	yes	yes	yes	yes	**5**

ANSWERS

Look after you have written something down. A wrong table you built is worth more than a right one you read.

Part One: the two-value machine

1.0a (a) 1, which is NOT. (b) 0, which is AND. (c) 0, which is OR. (d) 0, which is AND again: on 0 and 1, min and product are the same.
1.0b 0->0 gives max(1,0)=1. 0->1 gives max(1,1)=1. 1->0 gives max(0,0)=**0**. 1->1 gives max(0,1)=1. Only the third row, a true and b false, gives 0.
1.0c Grass is purple is false, so 0. Fish can knit is false, so 0. max(1-0, 0) = 1. **True.** And it tells you nothing, which is a separate fact.
1.0d It would need a relevance filter: a rule that the two sides must share content. Classical logic has no such setting, because a claim is one number and a number is not about anything.
1.1 Two rows: a is T gives F, a is F gives T.
1.2 Every row is F. It is never true, whatever a is. This is the law of non-contradiction.
1.3 Every row is T. It is always true. This is excluded middle, and Part Two shows it failing.
1.4 Yes, they match in all four rows. That is De Morgan.
1.5 If there is no smoke, the alarm did not sound.
1.6 Converse: if there is smoke, the alarm sounds. A case where the original is true and the converse false: the alarm is broken, so smoke with no sound. The original only promised sound implies smoke.
1.7 True. Pigs do not fly, so the promise was never tested. It is also completely uninformative, and both of those are the case at once.

Validity

2.1 Modus ponens. Valid.
2.2 Affirming the consequent. Invalid. Bad row: bins went out because it was a bank holiday collection, and it is Wednesday.
2.3 Denying the antecedent. Invalid. Bad row: you did not revise and passed anyway, because the paper was easy.
2.4 Modus tollens. Valid.
2.5 For example: all fish are birds, all birds have wheels, so all fish have wheels. The wiring is fine, everything else is not.
2.6 For example: the moon is round, so Paris is in France. The conclusion is true and the argument does nothing to support it.

Quantifiers

3.1 Some swan is not white.
3.2 All meetings are not useful, that is, no meeting is useful.
3.3 Some door in the building is not locked.
3.4 True, vacuously. There are no fish to be otherwise. And it tells you nothing, which is the same pairing as 1.7.
3.5 Any case where the property holds throughout. All seven cats are black, so some cat is black is also true. Some does not mean not all.
3.6 The first says each person has one, and they can all be different. The second says one particular song is everybody's favourite. Swapping the order of the quantifiers changed the claim.

Many-valued

4.1 ? AND F is F, because F wins in the minimum. ? OR T is T, because T wins in the maximum.
4.2 NOT(? AND T) is NOT ? which is ?. (NOT ?) OR (NOT T) is ? OR F which is ?. They match. De Morgan survives in K3.
4.3 NOT ? is ?. Then ? OR ? is ?. In K3 theta cuts above ?, so it is not assertible, so **not assertible**. Excluded middle fails.
4.4 Every step is identical. Only the last line changes: in LP theta cuts lower, so ? IS assertible and the answer is yes. Nothing about the tables moved.

4.5 In B3, F AND ? is ?. In K3 it is F. B3 lets the middle spread through everything, because ? means meaningless rather than unknown, and nonsense is not rescued by a false neighbour.
4.6 LP, because the records contradict and you must keep both without the system concluding everything. You have given up modus ponens, so the alert engine cannot chain inferences freely and has to be built to notice that.

Modal graphs

5.1 A cycle with no self-loops. Not reflexive, not symmetric. Transitive fails too: w points to x and x points to y, but there is no w to y arrow. Euclidean fails. So none of the five, and the system is K.
5.2 Now reflexive, so T and D. Still not symmetric or transitive, so no B, no 4, no 5.
5.3 All five shapes hold, so T, D, 4, B and 5. That is S5.
5.4 w points to x only. p is marked at y, not x. So []p is false at w and <>p is false at w as well, because the only dot w sees is x and p is not there.
5.5 []p is **true**, vacuously: there are no dots to fail on. <>p is **false**: there are no dots to succeed on. Every box holds and no diamond does.
5.6 Not reflexive, because you cannot pass. That costs you T, so from 'every position I can reach has feature X' you may not conclude the current position has X.
5.7 No. T says if you believe it then it is true, and beliefs can be false. Use D, which only requires beliefs to be non-contradictory.

WHAT YOU NOW HAVE

The whole book on one page

a logic	three settings: V, G and theta
V	the value space. What states the machine may be in
G	the gradient. How those states combine, or how you step between situations
theta	**the constraint.** On values it says which are assertible; on arrows it says which networks are admissible. The setting nobody mentions, and it decides the most
a truth table	an exhaustive check, and a load-relief device
valid	the conclusion cannot fail while the premises hold. About the wiring, not the world
vacuous truth	a rule that passed by having nothing to fail on. Ask for the count
the conditional	a comparison: is a no bigger than b? Nothing about the two sides being related
relevance	a setting classical logic does not have. Not a rule it broke
explosion	one contradiction yields everything. The price of two values
a possible world	**a dot**
accessibility	**the arrows**
[]p	ALL the dots I point to
<>p	SOME dot I point to
a modal axiom	a setting of theta on the arrows, read off
K, T, S4, S5	one machine, theta tightened by degrees
S5	everyone sees everyone. 5 frames in 512

The four questions

These are the portable part. They work without any of the machinery and they will do more for you than remembering the tables.

1	**What is the value space, and where is theta cut?** Two systems with identical tables give opposite answers if theta sits in a different place.
2	**How many cases actually got checked?** A rule that passed on zero cases carries no information and looks exactly like one that passed on a thousand.
3	**Possible relative to what?** Name the arrows. If nobody names them, the argument is not finished.
4	**Did the graph change halfway through?** Cheap possible, meaning I cannot rule it out, quietly becoming expensive possible, meaning there is genuinely a dot there.

What is not here

Predicate logic in full, with proofs and models. Natural deduction and sequent calculus. Intuitionistic logic, which drops excluded middle for a completely different reason from K3. Relevance logic. Fuzzy logic, which has infinitely many values rather than three. Second-order logic, completeness and compactness, and the whole of model theory.

Also not here: proof, in the strict sense. This book **computes** and the computations are exhaustive over the stated range, which is a genuine kind of evidence and is not the same as a general proof.

The last thing

Every logic in this book is a tool somebody built for a job. None was found. None is the logic. When one of them tells you something surprising about the world, the useful question is not whether the reasoning was valid, because it probably was.

The useful question is **what went in**.

A logic will faithfully compute the consequences of whatever you hand it, including a graph you invented and an object nobody is holding together. It has no way to tell you that you made those up. That part was always your job.